

MCT-454L Mobile Robotics

Lab Manual #03

Submitted To:
Dr. Maria Akram

Submitted By:
Muhammad Haider Ali (2022-MC-45)

1. Objectives

- Familiarize with ROS 2 workspaces and command-line interface (CLI) tools.
- Learn how to use GitHub for version control in a ROS 2 workspace.
- Understand the creation of a ROS 2 Python package and executable node.
- Develop Python nodes to move turtlesim in triangular, circular, and sinusoidal patterns.
- Learn to teleport a turtle to a specific location using ROS 2 services.

2. Prerequisites

- ROS 2 Humble installed on the system.
- Familiarity with Linux commands and terminal navigation.
- Understanding of ROS 2 nodes, topics, and services.
- Turtlesim simulator installed for testing motion control nodes.

3. Lab Setup

- Opened terminal on ROS 2-enabled machine.
- Created a ROS 2 workspace:

```
mkdir -p ~/ros2_ws/src  
cd ~/ros2_ws  
colcon build
```

Sourced the workspace:

```
source install/setup.bash
```

- Configured Git, initialized repository, linked GitHub, and pushed first commit.
- Created a ROS 2 Python package named `my_turtle_package` with dependencies `rclpy` and `turtlesim`.
- Placed Python scripts in the package's directory and set executable permissions.

4. Step-by-Step Instructions and Observations

Step 1: Launched Turtlesim

- Command: `ros2 run turtlesim turtlesim_node`
- Observation: Single turtle appears in center of blue background.

Step 2: Exploring ROS 2 Topics

Commands:

- `ros2 topic list` → Lists active topics.
- `ros2 topic echo /turtle1/pose` → Shows turtle1 position updates.

Observation: Turtle1's position and orientation are published continuously.

Step 3: Controlling Turtle1 via Velocity Node

- Python node: `VelocityPublisher`
- Behavior: Turtle1 moves in a triangular pattern using linear and angular velocities.
- Observation: Turtle follows the intended triangular path.

Step 4: Spawning and Controlling Turtle2

- Service: `/spawn`
- Coordinates: (5,5), name: `turtle1`, `turtle2`
- Command example:

```
ros2 service call /spawn turtlesim/srv/Spawn "{x: 5.0, y: 5.0, theta: 0.0, name: 'turtle2'}
```

- Observation: Turtle2 spawns at specified coordinates and moves in a circular pattern using a Python node.

Step 5: Spawning and Controlling Turtle3

- Service: `/spawn`
- Coordinates: (8,8), name: `turtle3`
- Behavior: Turtle3 moves in a sinusoidal path using `math.sin()` for angular velocity.

Observation: Turtle3 successfully follows wavy trajectory independently.

Step 6: Teleporting Turtle1 for Screenshot

- Service: /turtle1/teleport_absolute
- Command example:

```
ros2 service call /turtle1/teleport_absolute turtlesim/srv/TeleportAbsolute "{x: 5.5, y: 5.5, theta: 0.0}"
```

- Observation: Turtle1 instantly moved to target location, allowing clean screenshots.

5. Observations

- ROS 2 topics allow real-time monitoring of turtle positions and velocities.
- ROS 2 services such as /spawn and /teleport_absolute allow direct manipulation of turtle positions and creation of new turtles.
- Python nodes enable automation of complex motion patterns like triangular, circular, and sinusoidal paths.
- Multiple turtles can move independently and simultaneously without interfering with each other.

6. Problems Faced

- `turtlesim_node` did not launch initially → resolved by correctly sourcing ROS 2 environment.
- Spawning turtles required correct input for coordinates, orientation, and unique names.
- Synchronizing multiple turtles required managing separate publishers and timers for each turtle.
- Git push failed because local branch was behind remote: fixed using `git pull --rebase`.

7. Conclusion

- Lab reinforced practical knowledge of ROS 2 workspaces, nodes, topics, and services. Version control using Git and GitHub integrated well with the ROS 2 workflow.
- Successfully controlled multiple turtles in triangular, circular, sinusoidal, and absolute-positioned motion patterns.
- Objectives of Lab Manual #03 were fully achieved, demonstrating both fundamental ROS 2 operations and advanced control strategies for simulated mobile robots.

8. Screenshots Reference

All screenshots are located in `screenshots/` directory with respective step names and task names.